EFFECTS OF HIGH DIMENSIONALITY AND IMBALANCED DATA ON AN OPTIMAL MODEL FOR COLON CANCER DETECTION

Report submitted to the SASTRA Deemed to be University
as the requirement for the course

CSE300 / INT300 / ICT300 - MINI PROJECT

Submitted by
CHANDRAMOULI M
(Reg. No.: 126156022, B.Tech. Computer Science & Engineering (AI & DS))
DEEBADHARSAN K
(Reg. No.: 126156030, B.Tech. Computer Science and Engineering (AI & DS))
TEJAS SHIVA M
(Reg. No.: 126156164, B.Tech. Computer Science and Engineering (AI & DS))

May 2025

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)
THINK MERIT | THINK TRANSPARENCY | THINK SASTRA
THANJAVUR | KUMBAKONAM | CHENNAI

SCHOOL OF COMPUTING

THANJAVUR, TAMIL NADU, INDIA – 613 401

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

SCHOOL OF COMPUTING

THANJAVUR – 613 401

Bonafide Certificate

This is to certify that the report titled **"Effects of High Dimensionality and Imbalanced Data on an Optimal Model for Colon Cancer Detection"** submitted as a requirement for the course, CSE300 / INT300 / ICT300: MINI PROJECT for B.Tech. is a bonafide record of the work done by **Mr. Chandramouli M (Reg. No.: 126156022, B.Tech. Computer Science and Engineering (AI & DS)), Mr. Deebadharsan K (Reg. No.: 126156030, B.Tech. Computer Science and Engineering (AI & DS)), Mr. Tejas Shiva M (Reg. No.: 126156164, B.Tech. Computer Science and Engineering (AI & DS))** during the academic year 2024-25, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation :

Date :

Mini Project *Viva voc*e held on _____

Examiner 1 **Examiner 2**

Acknowledgements

We would like to thank our Honorable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honorable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan,** Dean**,** Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli,** Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K.Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare

Our guide **Dr. B. Thamotharan**, Assistant Professor-III, School of Computing was the driving force behind this whole idea from the start. His deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing us an opportunity to showcase our skills through project.

List of Figures

Fig No.	Title	Page No.
4.0	The Class Distribution Bar Graph Before and After Balancing the Dataset	11
4.1	Comparison of Individual Models with All Features	11
4.2	Comparison of Individual Models after Feature Selection using SU	12
4.3	Comparison of Individual Models after Feature Selection using Chi-Square	13
4.4	Comparison of Individual Models after Feature Selection using CFS	13
4.5	Accuracy Comparison of Individual Models on Imbalanced Dataset	14
4.6	Accuracy Comparison of Individual Models on Balanced Dataset	15
4.7	Overall Comparison of Classification Accuracy of Individual ML Models	15
4.8	Comparison of Ensemble Models with All Features	16
4.9	Comparison of Ensemble Models with Feature Selection using SU	17
4.10	Comparison of Ensemble Models with Feature Selection using Chi-Square	17
4.11	Comparison of Ensemble Models with Feature Selection using CFS	18
4.12	Accuracy Comparison of Ensemble Models on Imbalanced Dataset	19
4.13	Accuracy Comparison of Ensemble Models on Balanced Dataset	19
4.14	Overall Comparison of Classification Accuracy of Ensemble Models	20
4.15	Comparison of Classification Accuracy of Optimized SVM-KNN-RF Model	21
4.16	Evaluation of Ensemble Model on Imbalanced Dataset– Effect of GA	21
4.17	Evaluation of Ensemble Model on Balanced Dataset– Effect of GA	22
4.18	Effect of GA on Classification Accuracy of SVM-KNN-RF Ensemble	23

List of Tables

Table No.	Table name	Page No.
4.1	Accuracies of Individual Models with All Features	11
4.2	Accuracies of Individual Models with SU	12
4.3	Accuracies of Individual Models with Chi-Square	12
4.4	Accuracies of Individual Models with CFS	13
4.5	Accuracies of Individual Models on Imbalanced Dataset	14
4.6	Accuracies of Individual Models on Balanced Dataset	14
4.7	Classification Accuracy of Individual ML Models	15
4.8	Accuracies of Ensemble Models with All Features	16
4.9	Accuracies of Ensemble Models with SU	16
4.10	Accuracies of Ensemble Models with Chi-Square	17
4.11	Accuracies of Ensemble Models with CFS	18
4.12	Accuracies of Ensemble Models on Imbalanced Dataset	18
4.13	Accuracies of Ensemble Models on Balanced Dataset	19
4.14	Classification Accuracy of Ensemble Models	20
4.15	Classification Accuracy of Optimized SVM-KNN-RF Model	20
4.16	Accuracy of Ensemble before and after GA on Imbalanced Dataset	21
4.17	Accuracy of Ensemble before and after GA on Balanced Dataset	22
4.18	Effect of GA on Classification Accuracy of SVM-KNN-RF Ensemble	22

Abbreviations

Abbreviation	Full Form
CFS	Correlation-based Feature Selection
GA	Genetic Algorithm
KNN	K-Nearest Neighbor
RF	Random Forest
SMOTE	Synthetic Minority Over-sampling Technique
SU	Symmetrical Uncertainty
SVM	Support Vector Machine

Notations

English Symbols (in alphabetical order)

k	Number of nearest neighbors in KNN
n	Number of samples in the dataset
p	Number of features/attributes
w	Weight assigned to a classifier in ensemble
x	Input feature vector
y	Target/output variable (class label)

Abstract

The effectiveness of traditional machine learning methods in detecting colon cancer at an early stage is often limited due to challenges associated with high-dimensional datasets and imbalanced class distributions. This project explores the application of a hybrid-optimized model, originally designed for medical text classification, to address these issues in colon cancer detection.

The proposed approach integrates three machine learning algorithms: Support Vector Machines (SVM), Random Forest (RF), and Logistic Regression (LR) into a soft voting ensemble framework. To improve the model's performance, an adaptive genetic algorithm (GA) is employed to optimize the weight parameters of each classifier, ensuring a balanced contribution from each model.
The dataset used in this study is a publicly available colon cancer dataset characterized by high-dimensional gene expression data. Given the imbalance in the dataset, the Synthetic Minority Oversampling Technique (SMOTE) is applied to balance the class distribution. Additionally, feature selection techniques such as Symmetrical Uncertainty (SU) and Correlation-Based Feature Selection (CFS) are used to identify the most relevant features while eliminating noise and redundancy in the data.

The approach is designed to enhance prediction accuracy and demonstrate the potential for expanding the use of machine learning techniques in addressing critical challenges in medical prediction tasks. This project aims to use hybrid models to improve detection systems in complex datasets and provide a foundation for extending such models to other medical prediction problems.

KEY WORDS: *Ensemble model, High-dimensional data, Imbalanced class distribution, Feature selection.*

Table of Contents

Title	Page No.
Bonafide Certificate	ii
Acknowledgements	iii
List of Figures	iv
List of Tables	v
Abbreviations	vi
Notations	vii
Abstract	viii
1. Summary of the base paper	1
2 Merits and Demerits of the base paper	3
3 Sample Source Code	5
4 Results	11
5 Conclusion and Future Plans	24
6 References .	25

CHAPTER 1

SUMMARY OF THE BASE PAPER

Title:

Exploring the Effects of High Dimensionality and Imbalanced Data on Predictive Models for Colon Cancer Detection Using Tree, Rule, and Lazy Learning Techniques

Journal: Journal of Artificial Intelligence and Technology

Publisher: ISTP Press

Year: 2024

Indexed in: Scopus

DOI: https://doi.org/10.37965/jait.2024.0581

Tambe S.N. et al. (2024) addresses a critical challenge in the field of medical data analysis: the impact of high-dimensionality and class imbalance on the performance of machine learning models for colon cancer detection. Colon cancer datasets typically consist of thousands of gene expression features but relatively few patient samples, leading to difficulties in building robust predictive models. Additionally, the imbalance between healthy and cancerous samples can bias classifiers toward the majority class, reducing sensitivity and overall accuracy.

Contribution:

- The paper presents a comprehensive framework that systematically addresses both high-dimensionality and class imbalance in the colon cancer dataset.
- It evaluates the impact of various feature selection techniques (Chi-Square, SU, CFS, and classifier subset evaluators) on the performance of several popular classification algorithms, including tree-based (J48, Simple Cart), rule-based (Jrip, Ridor), lazy (KNN), and Support Vector Machine (SVM).
- The study is among the first to provide a comparative analysis of these techniques specifically for colon cancer detection, demonstrating how preprocessing steps can significantly enhance model accuracy.

Research Problem:

How do high-dimensionality and class imbalance affect the predictive accuracy of machine learning models for colon cancer detection, and what preprocessing strategies can mitigate these effects?

Solution Proposed:

The authors propose a multi-step framework:

1. **Data Preprocessing:**
    - Address class imbalance using resampling techniques to ensure a more equitable class distribution.
    - Apply feature selection methods to reduce dimensionality and retain only the most informative gene expression features.
2. **Model Training and Evaluation:**
    - Train multiple classifiers (tree, rule, lazy, and SVM) on both the original and preprocessed datasets.
    - Evaluate the performance of each model using accuracy as the primary metric, comparing results across different feature selection methods and class balance scenarios.

Key Findings:

- All classifiers showed improved performance after balancing the dataset and applying feature selection.
- Among the methods tested:
    - **SVM achieved the highest accuracy (92.85%) with Symmetrical Uncertainty feature selection.**
    - **KNN reached 91.66% accuracy with Chi-Square and CFS feature selection.**
    - **Rule-based and tree-based methods also saw significant improvements, with Jrip, Ridor, and J48 achieving over 83% accuracy in the best cases.**
- The study confirms that careful preprocessing-balancing and feature selection-is crucial for maximizing the predictive power of machine learning models in high-dimensional, imbalanced medical datasets.

Conclusion:

The base paper makes a significant contribution to the field of medical machine learning by systematically evaluating how high-dimensionality and class imbalance affect colon cancer detection models and by proposing a robust preprocessing and modeling framework. The findings underscore the importance of balancing and feature selection in achieving high accuracy, with SVM and KNN classifiers performing best when paired with appropriate feature selection techniques. The study's methodology and results provide a valuable reference for future research on predictive modeling in complex biomedical datasets.

CHAPTER 2

MERITS AND DEMERITS OF THE BASE PAPER

1. Existing Techniques

- **Traditional Machine Learning Models:**
  Methods such as SVM, RF, KNN, DT, and LR have been widely used. These models require careful feature engineering and are sensitive to high dimensionality and class imbalance.
- **Deep Learning Approaches:**
  Recent works have explored CNN, ANN, and hybrid models for colon cancer screening and classification. Deep learning models can achieve high accuracy but often lack interpretability and require large, well-annotated datasets.
- **Data Balancing Techniques:**
  Oversampling methods like SMOTE and under-sampling strategies such as ENN and RENN are commonly used to address class imbalance. Hybrid sampling methods like RENN + SMOTE have been proposed to balance sensitivity and specificity, though at the cost of higher computational demands.
- **Feature Selection Methods:**
  Statistical tests (e.g., t-test, Chi-Square), SU, CFS, and PCA are frequently applied to reduce dimensionality and improve model performance.

2. Merits of the Proposed Techniques in the Base Paper

- **Comprehensive Framework:**
  The base paper presents a unified framework that systematically addresses both high dimensionality and class imbalance, two major challenges in colon cancer data analysis.
- **Extensive Comparative Analysis:**
  The study evaluates multiple classifiers (tree, rule, lazy/KNN, SVM) and feature selection methods (Chi-Square, SU, CFS, classifier subset evaluators), providing a thorough comparison across preprocessing strategies and model types.
- **Demonstrated Performance Gains:**
  After balancing the dataset and applying feature selection, all classifiers showed improved accuracy. For example, SVM achieved 92.85% accuracy with SU on a balanced dataset, and KNN reached 91.66% accuracy with Chi-Square and CFS.
- **Addressing Preprocessing Importance:**
  The research highlights the critical role of preprocessing in maximizing model effectiveness, which is often underemphasized in other works.
- **Applicability to Real-World Data:**
  The framework is validated on a high-dimensional, real-world colon cancer gene expression dataset, demonstrating its practical relevance.
- **Balanced Evaluation:**
  The study carefully compares results on both imbalanced and balanced datasets, providing insights into the effect of class distribution on model performance.

3. Demerits and Limitations of the Proposed Techniques

- **Computational Complexity:**
  The use of multiple feature selection methods increases computational cost and complexity. This may limit scalability to very large datasets or real-time applications.
- **Limited Deep Learning Exploration:**
  While the paper covers a broad range of classical machine learning models, it does not evaluate deep learning approaches, which have shown promise in recent studies for feature extraction and classification in genomics.
- **Generalizability:**
  The framework is validated on a specific colon cancer dataset; its performance and robustness on other datasets or cancer types remain to be demonstrated.
- **Resource Requirements:**
  The need for extensive preprocessing and optimization steps may require significant computational resources and expertise, which could be a barrier for some research or clinical environments.

CHAPTER 3

SAMPLE SOURCE CODES

1) Import Necessary Modules:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, KBinsDiscretizer
from imblearn.over_sampling import SMOTE
from collections import Counter
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.feature_selection import mutual_info_classif
from sklearn.base import clone
from scipy.stats import chi2_contingency, pearsonr
from itertools import combinations
import random
from sklearn.impute import SimpleImputer

random.seed(42)
np.random.seed(42)
```

2) Load Data:

```
original_df = pd.read_csv("Colon Cancer Datset.csv")
df = original_df.copy()
X = df.drop(columns=["samples", "response"])
y = df["response"]

# Encode target
le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Discretize features into 5 ordinal bins
n_bins = 5
discretizer = KBinsDiscretizer(n_bins=n_bins, encode='ordinal',
strategy='quantile')
X_discretized = discretizer.fit_transform(X)
```

3) Handling Imbalance:

```
# Identify the minority class (to oversample only 'normal' class)
class_counts = Counter(y_encoded)
minority_class = min(class_counts, key=class_counts.get)
majority_class = max(class_counts, key=class_counts.get)

# Apply SMOTE only to the minority (assumed "normal") class
smote = SMOTE(sampling_strategy={minority_class: class_counts[minority_class]
* 2}, random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y_encoded)
```

4) Chi – Square Feature Selection:

```
chi2_scores = []
for i in range(X_discretized.shape[1]):
    feature = pd.Series(X_discretized[:, i], name=f'V{i+1}')
    contingency_table = pd.crosstab(feature, y_encoded)
    chi2_stat, _, _, _ = chi2_contingency(contingency_table)
    chi2_scores.append(chi2_stat)

feature_scores = pd.DataFrame({
    'Feature': X.columns,
    'Chi2_Score': chi2_scores
}).sort_values('Chi2_Score', ascending=False)

# Reduce to top features and create filtered DataFrame
ChiSq_imb_df = df[feature_scores['Feature'][:57].tolist()]
```

5) SU:

```
# Calculate SU scores and create a DataFrame to store the scores
su_scores = mutual_info_classif(X, y_encoded, discrete_features='auto')
su_df = pd.DataFrame({'Feature': X.columns, 'SU Score': su_scores})

# Filter features with SU score > 0
filtered_features = su_df[su_df['SU Score'] > 0]['Feature'].tolist()

# Create a new DataFrame with only the selected features
X_filtered = X[filtered_features]
su_balanced_df = X[filtered_features[:12]]
```

6) CFS:

```
# Convert to DataFrame for correlation calculations
X_encoded = X.copy()
for col in X_encoded.columns:
    if X_encoded[col].dtype == 'object':
        X_encoded[col] = LabelEncoder().fit_transform(X_encoded[col])
X_encoded = pd.DataFrame(X_encoded, columns=X.columns)

# Function to compute CFS merit
def cfs_merit(subset, X, y):
    k = len(subset)
    if k == 0:
        return 0

    # Average correlation with class
    cor_with_class = [abs(pearsonr(X[feat], y)[0]) for feat in subset]
    avg_cor = np.mean(cor_with_class)

    # Average feature-feature correlation
    if k == 1:
        avg_red = 0  # no redundancy if only one feature
    else:
        red = []
        for f1, f2 in combinations(subset, 2):
            red.append(abs(pearsonr(X[f1], X[f2])[0]))
        avg_red = np.mean(red)
    merit = (k * avg_cor) / np.sqrt(k + k * (k - 1) * avg_red)
    return merit

# Greedy forward selection using CFS merit
remaining_features = list(X_encoded.columns)
selected_features = []
best_merit = 0
feature_scores = []

while remaining_features:
    merits = []
    for feat in remaining_features:
        candidate_set = selected_features + [feat]
        merit = cfs_merit(candidate_set, X_encoded, y_encoded)
        merits.append((feat, merit))
```

```
    # Select the feature with the highest merit improvement
    best_feat, merit = max(merits, key=lambda x: x[1])
    if merit > best_merit:
        selected_features.append(best_feat)
        best_merit = merit
        feature_scores.append((best_feat, best_merit))
        remaining_features.remove(best_feat)
    else:
        break  # no improvement, stop

# Convert to DataFrame and sort
cfs_scores_df = pd.DataFrame(feature_scores, columns=["Feature", "CFS_Merit"])
cfs_scores_df = cfs_scores_df.sort_values(by="CFS_Merit", ascending=False)

print(cfs_scores_df)
```

7) Selection of Number of Features for Best Accuracy :

```
models = {
    'SVM': SVC(kernel='linear', probability=True),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

results = {}
for i in range(1, 2001):
    res_subset = {}
    selected_features = feature_scores['Feature'][:i].tolist()
    filtered_df = df[selected_features]
    X_train, X_test, y_train, y_test = train_test_split(filtered_df, y, test_size=0.2,
random_state=42)
    for modelname, model in models.items():
        model.fit(X_train, y_train)
        y_pred_test = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred_test)
        res_subset[modelname] = acc
    results[i] = res_subset
# Remove duplicate accuracy results
seen_values = set()
keys_to_delete = []
for key, value in results.items():
    value_tuple = tuple(value.values())
    if value_tuple in seen_values:
        keys_to_delete.append(key)
```

```
    else:
        seen_values.add(value_tuple)
for key in keys_to_delete:
    del results[key]

# Sort keys by model performance
svm_keys = sorted([key for key in results if 'SVM' in results[key]], key=lambda x:
results[x]['SVM'], reverse=True)
rf_keys = sorted([key for key in results if 'Random Forest' in results[key]],
key=lambda x: results[x]['Random Forest'], reverse=True)
knn_keys = sorted([key for key in results if 'KNN' in results[key]], key=lambda x:
results[x]['KNN'], reverse=True)

print("SVM Reduced & Sorted Keys:", svm_keys)
print("KNN Reduced & Sorted Keys:", knn_keys)
print("RF Reduced & Sorted Keys:", rf_keys)
```

8) Ensemble Model:

```
model_combinations = [
    ('svm', svm),
    ('knn', knn),
    ('rf', rf),
    ('svm_knn', VotingClassifier(estimators=[('svm', svm), ('knn', knn)], voting='soft',
weights=[0.5, 0.5])),
    ('svm_rf', VotingClassifier(estimators=[('svm', svm), ('rf', rf)], voting='soft',
weights=[0.5, 0.5])),
    ('knn_rf', VotingClassifier(estimators=[('knn', knn), ('rf', rf)], voting='soft',
weights=[0.5, 0.5])),
    ('svm_knn_rf', VotingClassifier(estimators=[('svm', svm), ('knn', knn), ('rf', rf)],
voting='soft', weights=[0.33, 0.33, 0.33]))
]

for name, model in model_combinations:

  # Fit the model
  model.fit(X_train, y_train)

  # Make the Predictions
   y_pred = model.predict(X_test)

  # Evaluate the model
  accuracy = accuracy_score(y_test, y_pred)
  print(f"Model: {name}, Accuracy: {accuracy}")
```

9) GA:

```
def evaluate_weights(weights, models, X_val, y_val):
    weights = np.array(weights) / np.sum(weights)
    probas = np.zeros((X_val.shape[0], len(np.unique(y_val))))
    for weight, model in zip(weights, models):
        probas += weight * model.predict_proba(X_val)
    preds = np.argmax(probas, axis=1)
    return accuracy_score(y_val, preds)

def genetic_algorithm(models, X_val, y_val, generations=20, population_size=20,
mutation_rate=0.01):
    n_models = len(models)

    def create_individual():
        w = np.random.rand(n_models)
        return w / np.sum(w)
    def mutate(individual):
        idx = random.randint(0, n_models - 1)
        individual[idx] += np.random.uniform(-0.1, 0.1)
        individual = np.clip(individual, 0.01, 1.0)
        return individual / np.sum(individual)
    def crossover(parent1, parent2):
        point = random.randint(1, n_models - 1)
        child = np.concatenate([parent1[:point], parent2[point:]])
        return child / np.sum(child)

    population = [create_individual() for _ in range(population_size)]
    for _ in range(generations):
        scores = [evaluate_weights(ind, models, X_val, y_val) for ind in population]
        sorted_population = [ind for _, ind in sorted(zip(scores, population),
key=lambda item: item[0], reverse=True)]
        population = sorted_population[:population_size // 2]
        while len(population) < population_size:
            parent1, parent2 = random.sample(population[:5], 2)
            child = crossover(parent1, parent2)
            if random.random() < mutation_rate:
                child = mutate(child)
            population.append(child)
    return max(population, key=lambda ind: evaluate_weights(ind, models, X_val,
y_val))

# Optimize weights using genetic algorithm
optimized_weights = genetic_algorithm(models, X_val, y_val)
```

CHAPTER 4

RESULTS

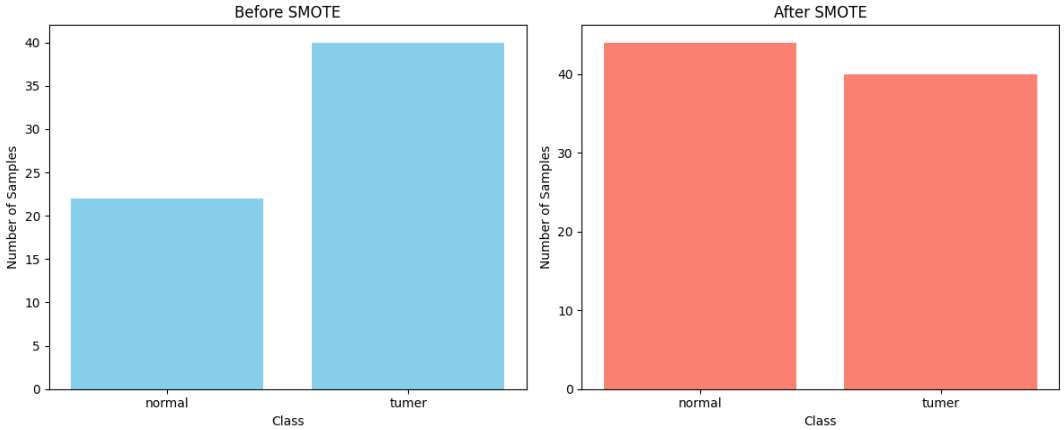

Fig 4.0 The Class Distribution Bar Graph Before and After Balancing the Dataset

1) Effect of Class Imbalance on Individual ML Models

Models	Imbalanced	Balanced
SVM	69.2	88.2
KNN	61.5	82.4
RF	76.9	88.2

Table 4.1 Accuracies of Individual Models with All Features

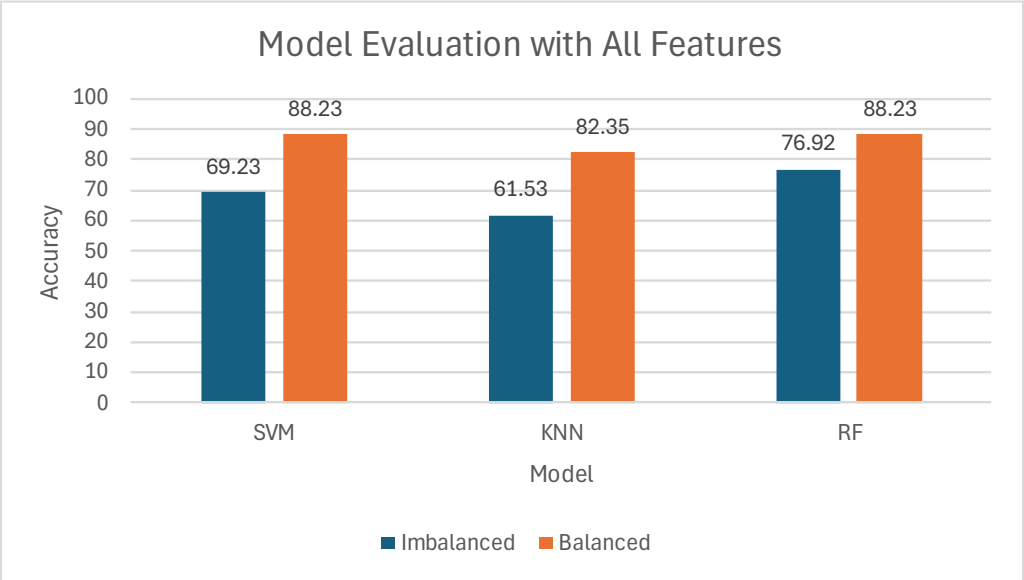

Fig 4.1 Comparison of Individual Models with All Features

Models	Imbalanced	Balanced
SVM	76.9	94.1
KNN	76.9	88.2
RF	76.9	94.1

Table 4.2 Accuracies of Individual Models with SU

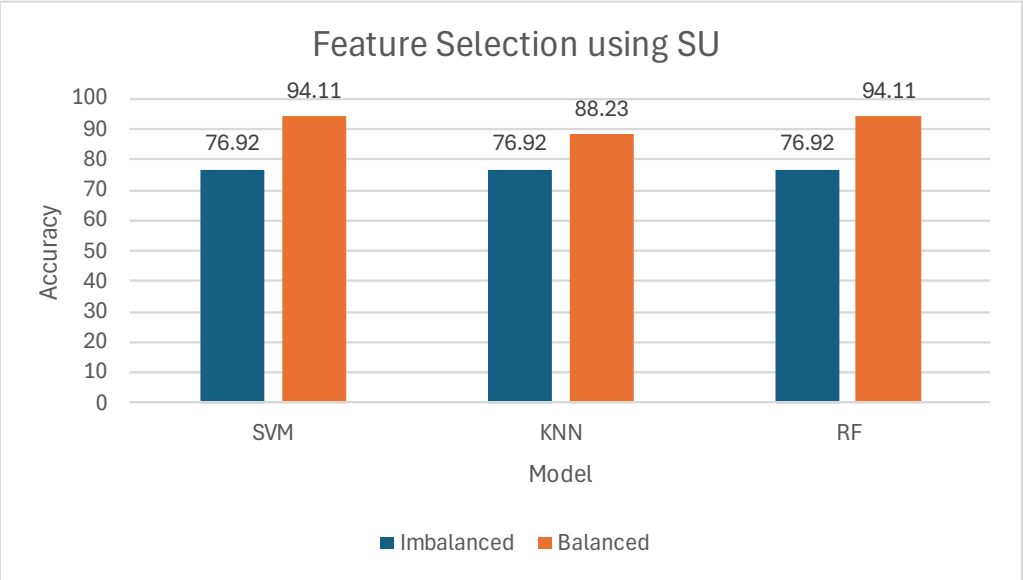

Fig 4.2 Comparison of Individual Models after Feature Selection using SU

Models	Imbalanced	Balanced
SVM	92.3	94.1
KNN	76.9	94.1
RF	84.6	94.1

Table 4.3 Accuracies of Individual Models with Chi-Square

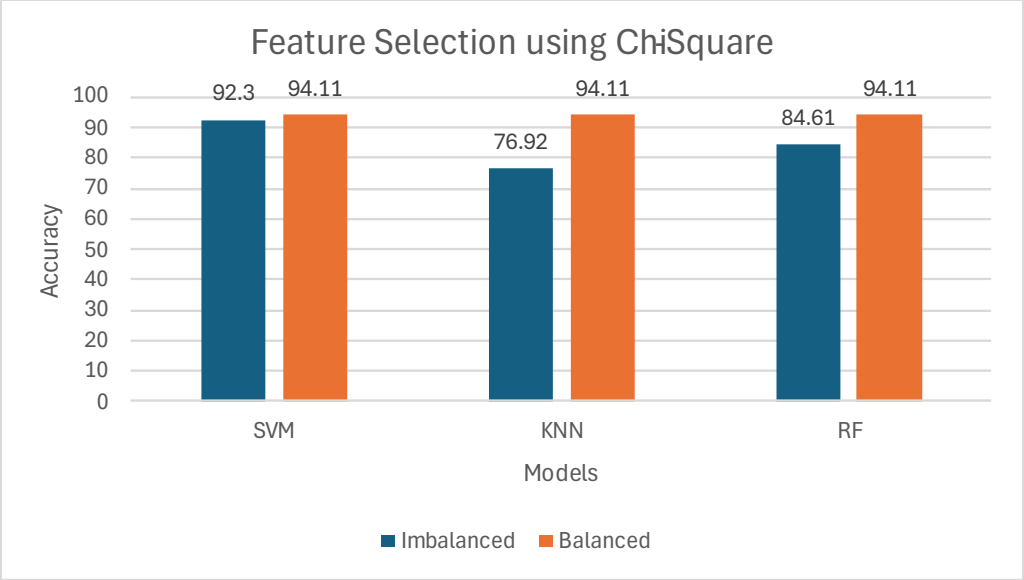

Fig 4.3 Comparison of Individual Models after Feature Selection using Chi-Square

Models	Imbalanced	Balanced
SVM	84.6	94.1
KNN	84.6	100
RF	76.9	88.2

Table 4.4 Accuracies of Individual Models with CFS

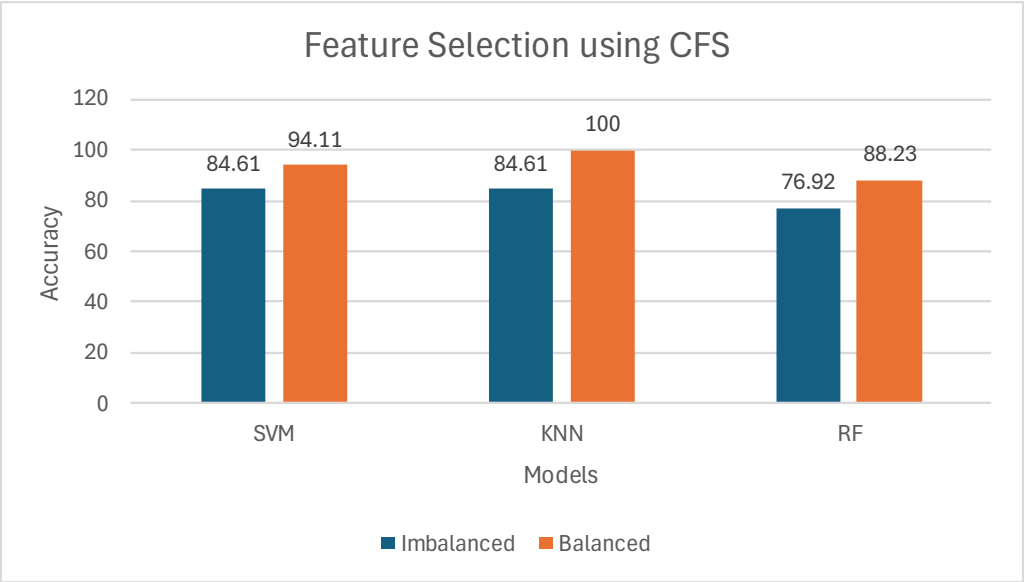

Fig 4.4 Comparison of Individual Models after Feature Selection using CFS

2) Effect of Various Feature Selection Methods on Individual ML Models

Models	SVM	KNN	RF
All Features	69.23	61.53	76.92
SU Top 157	76.92	76.92	76.92
Chi-Square Top 59	92.3	76.92	84.61
CFS Top 16	84.61	84.61	76.92

Table 4.5 Accuracies of Individual Models on Imbalanced Dataset

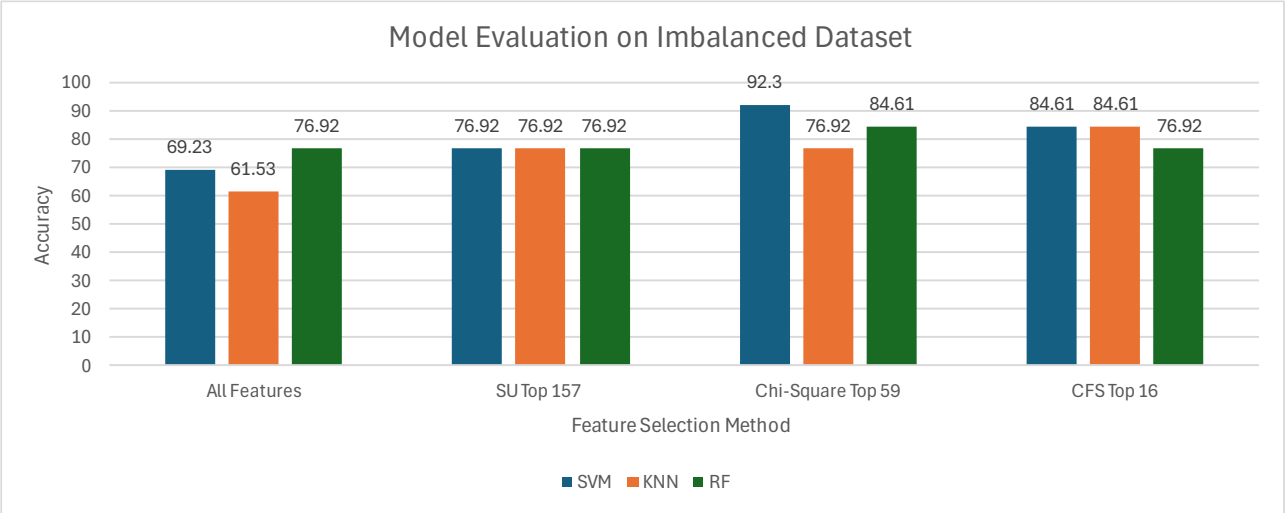

Fig 4.5 Accuracy Comparison of Individual Models on Imbalanced Dataset

Models	SVM	KNN	RF
All Features	88.23	82.35	88.23
SU Top 12	94.11	88.23	94.11
Chi-Square Top 10	94.11	94.11	94.11
CFS Top 19	94.11	100	88.23

Table 4.6 Accuracies of Individual Models on Balanced Dataset

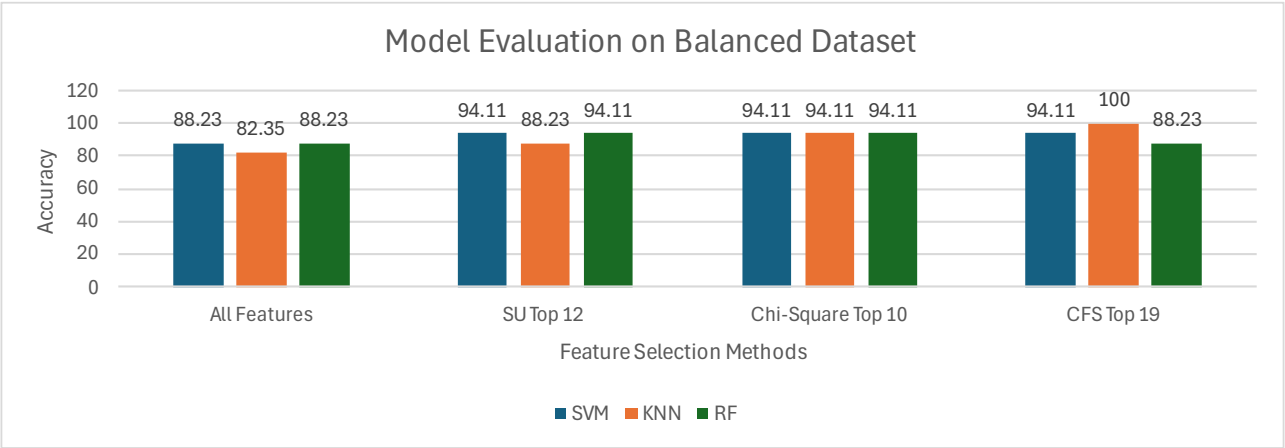

Fig 4.6 Accuracy Comparison of Individual Models on Balanced Dataset

3) Effect of High Dimensionality and Class Imbalance on Individual ML Models

Models	SVM		KNN		RF	
	Imbalanced	Balanced	Imbalanced	Balanced	Imbalanced	Balanced
All Features	69.23	88.23	61.53	82.35	76.92	88.23
SU	76.92	94.11	76.92	88.23	76.92	94.11
Chi Square	92.30	94.11	76.92	94.11	84.61	94.11
CFS	84.61	94.11	84.61	100.00	76.92	88.23

Table 4.7 Classification Accuracy of Individual ML Models

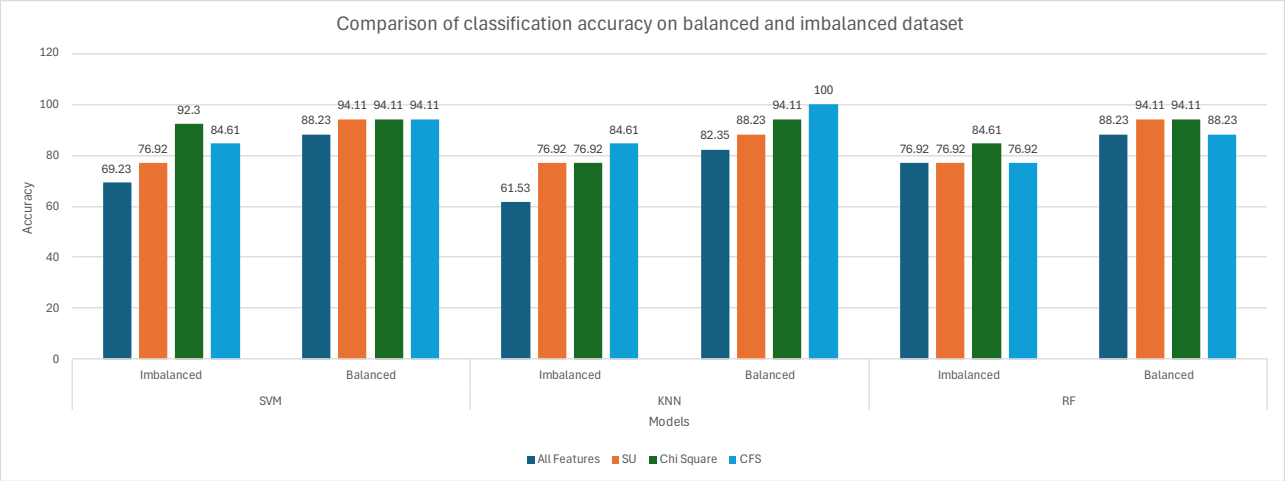

Fig 4.7 Overall Comparison of Classification Accuracy of Individual ML Models

4) Effect of Class Imbalance on Ensemble Models

Model	Imbalanced	Balanced
SVM – KNN	69.2	88.2
SVM – RF	69.2	88.2
KNN – RF	76.9	82.4
SVM – KNN - RF	76.9	88.2

Table 4.8 Accuracies of Ensemble Models with All Features

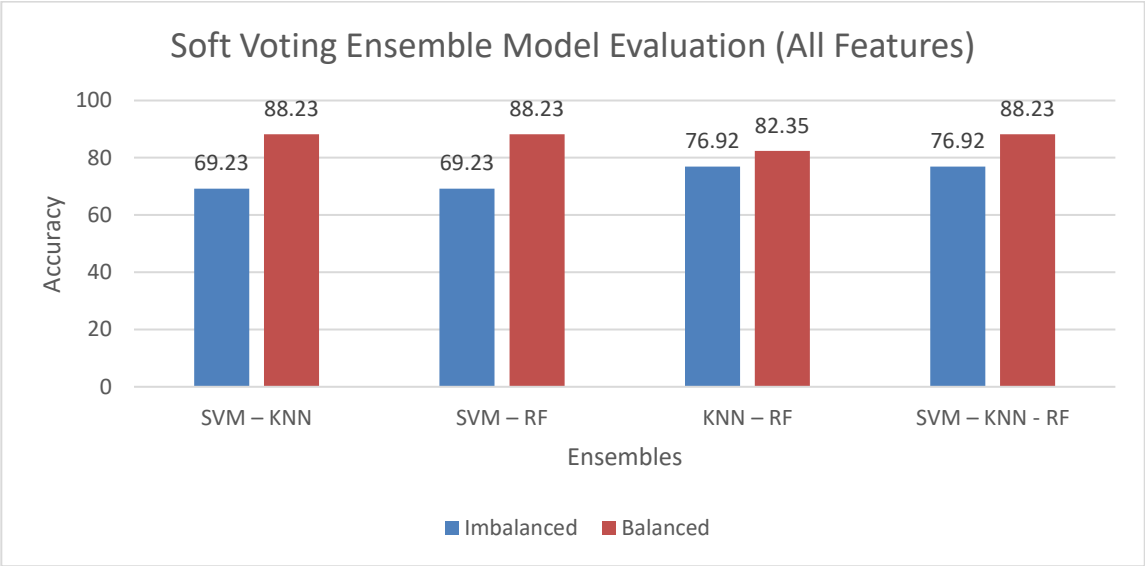

Fig 4.8 Comparison of Ensemble Models with All Features

Model	Imbalanced	Balanced
SVM – KNN	76.9	94.1
SVM – RF	76.9	94.1
KNN – RF	76.9	88.2
SVM – KNN - RF	76.9	94.1

Table 4.9 Accuracies of Ensemble Models with SU

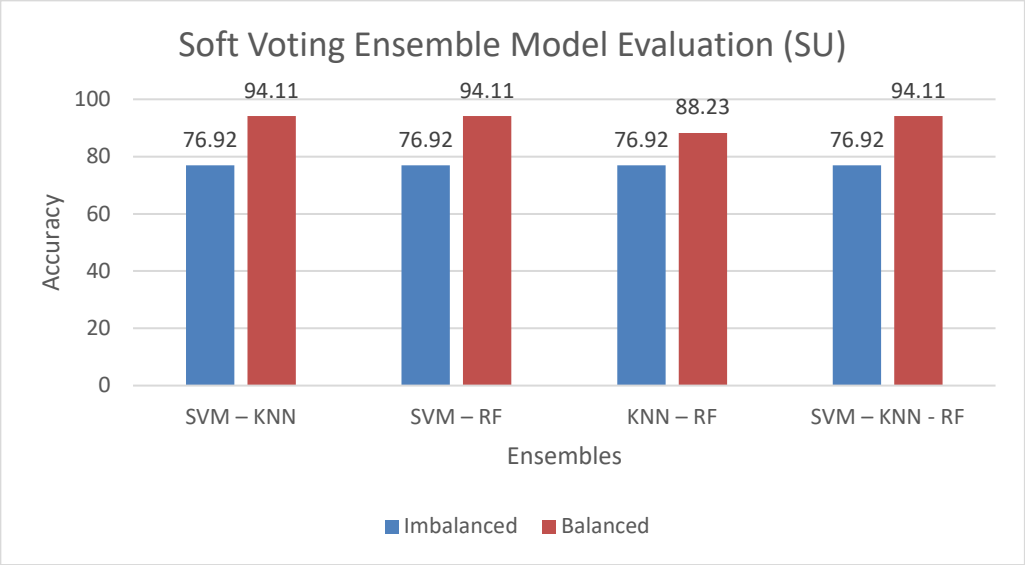

Fig 4.9 Comparison of Ensemble Models with Feature Selection using SU

Model	Imbalanced	Balanced
SVM – KNN	76.9	100
SVM – RF	84.6	94.1
KNN – RF	76.9	94.1
SVM – KNN - RF	76.9	100

Table 4.10 Accuracies of Ensemble Models with Chi-Square

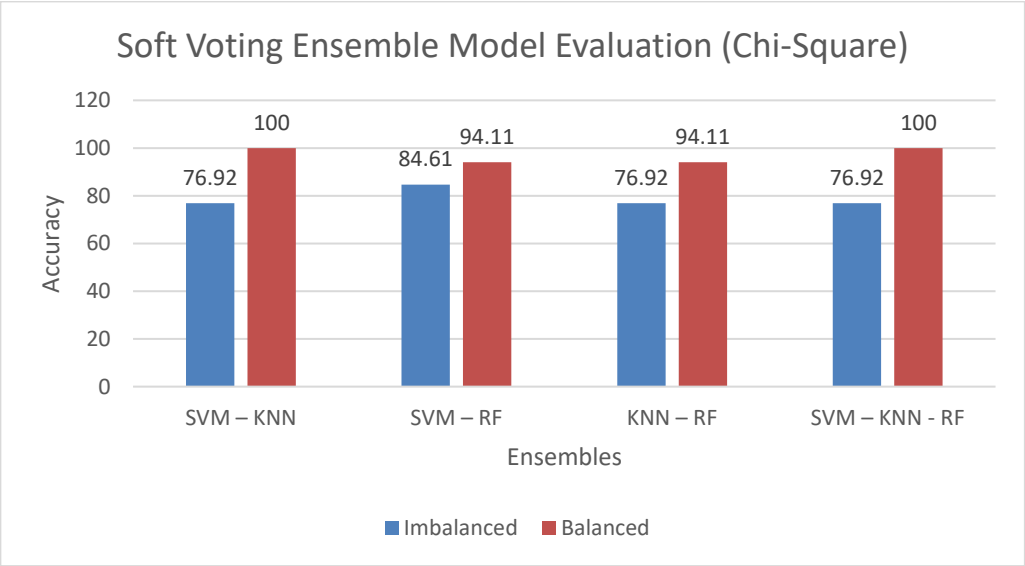

Fig 4.10 Comparison of Ensemble Models with Feature Selection using Chi-Square

Model	Imbalanced	Balanced
SVM – KNN	84.6	94.1
SVM – RF	84.6	94.1
KNN – RF	84.6	100
SVM – KNN - RF	84.6	94.1

Table 4.11 Accuracies of Ensemble Models with CFS

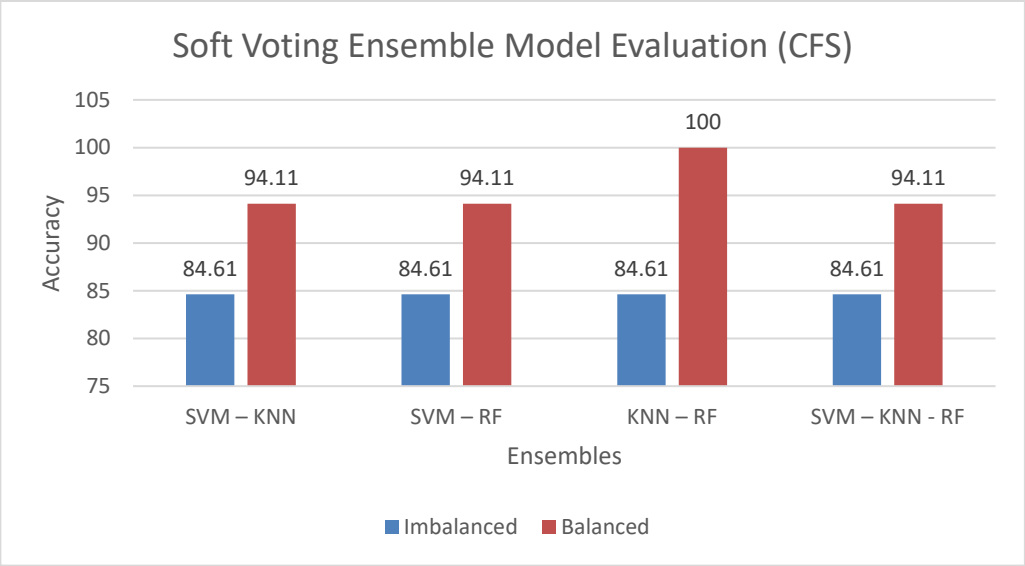

Fig 4.11 Comparison of Ensemble Models with Feature Selection using CFS

5) Effect of Various Feature Selection Methods on Ensemble Models

Models	SVM - KNN	SVM - RF	KNN - RF	SVM – KNN - RF
All Features	69.23	69.23	76.92	76.92
SU	76.92	76.92	76.92	76.92
Chi-Square	76.92	84.61	76.92	76.92
CFS	84.61	84.61	84.61	84.61

Table 4.12 Accuracies of Ensemble Models on Imbalanced Dataset

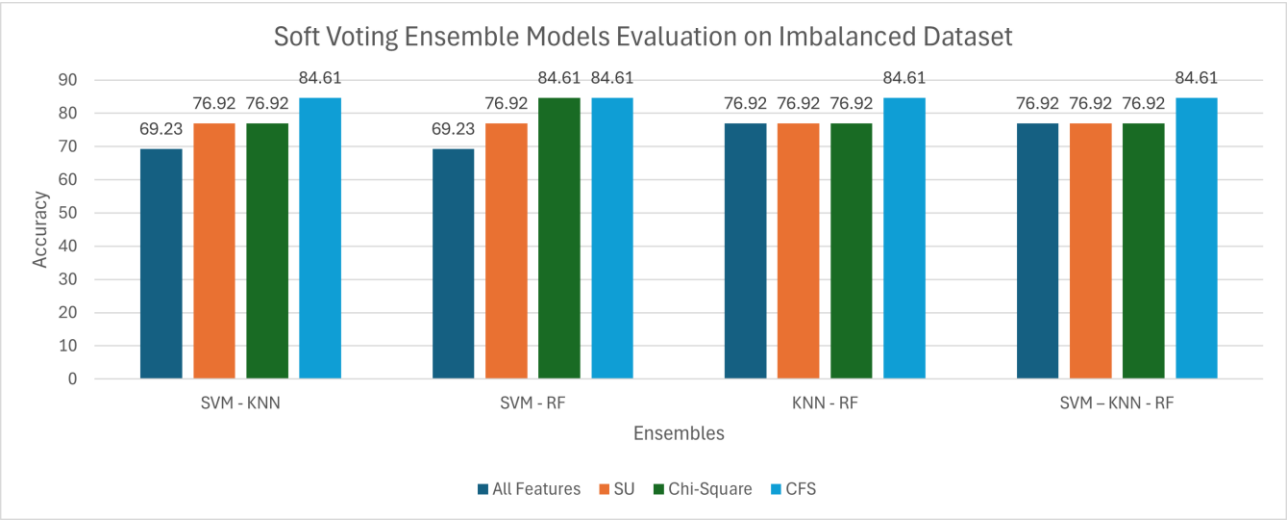

Fig 4.12 Accuracy Comparison of Ensemble Models on Imbalanced Dataset

Models	SVM - KNN	SVM - RF	KNN - RF	SVM – KNN - RF
All Features	88.23	88.23	82.35	88.23
SU	94.11	94.11	88.23	94.11
Chi-Square	100	94.11	94.11	100
CFS	94.11	94.11	100	94.11

Table 4.13 Accuracies of Ensemble Models on Balanced Dataset

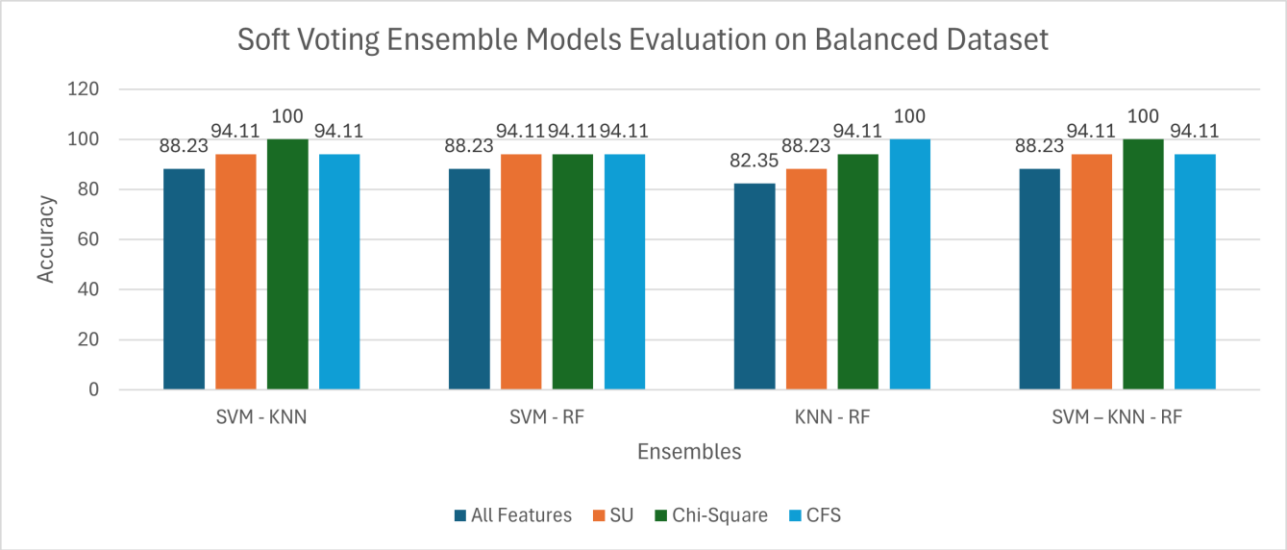

Fig 4.13 Accuracy Comparison of Ensemble Models on Balanced Dataset

6) Effect of High Dimensionality and Class Imbalance on Ensemble Models

	SVM – KNN		SVM – RF		KNN – RF		SVM – KNN – RF	
	Imbalanced	Balanced	Imbalanced	Balanced	Imbalanced	Balanced	Imbalanced	Balanced
All	69.23	88.23	69.23	88.23	76.92	82.35	76.92	88.23
SU	76.92	94.11	76.92	94.11	76.92	88.23	76.92	94.11
Chi	76.92	100	84.61	94.11	76.92	94.11	76.92	100
CFS	84.61	94.11	84.61	94.11	84.61	100	84.61	94.11

Table 4.14 Classification Accuracy of Ensemble Models

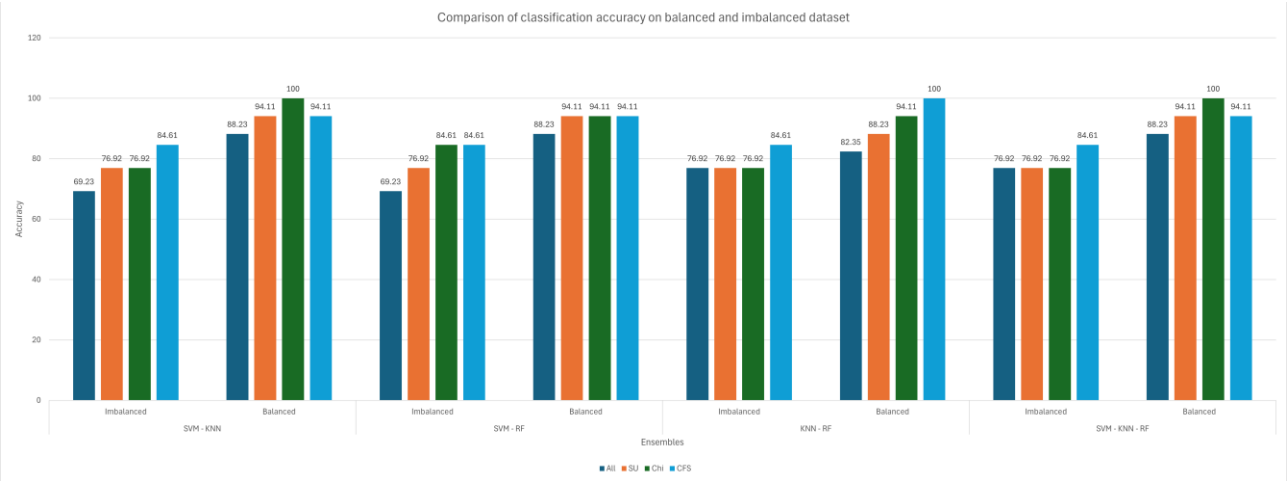

Fig 4.14 Overall Comparison of Classification Accuracy of Ensemble Models

7) Weight Optimization using GA on SVM-KNN-RF Ensemble

Features	Imbalanced	Balanced
All Features	84.62	93.75
SU	84.62	93.75
Chi-Square	76.92	93.75
CFS	92.3	100

Table 4.15 Classification Accuracy of Optimized SVM-KNN-RF Model

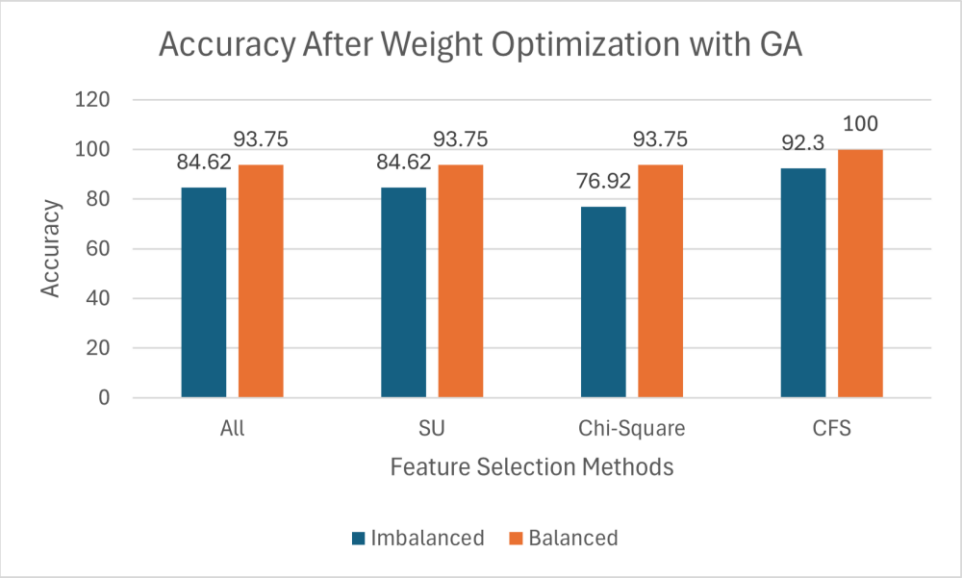

Fig 4.15 Comparison of Classification Accuracy of Optimized SVM-KNN-RF Model

8) Effect of GA on SVM-KNN-RF Ensemble

Features	Before GA	After GA
All Features	76.92	84.62
SU	76.92	84.62
Chi-Square	76.92	76.92
CFS	84.61	92.3

Table 4.16 Accuracy of Ensemble before and after GA on Imbalanced Dataset

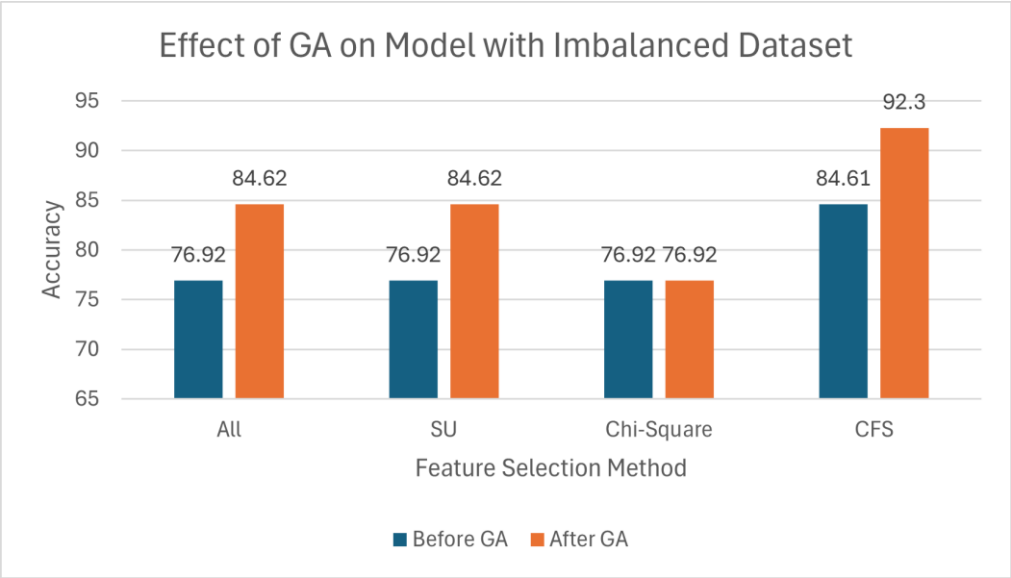

Fig 4.16 Evaluation of Ensemble Model on Imbalanced Dataset– Effect of GA

Features	Before GA	After GA
All Features	88.23	93.75
SU	94.11	93.75
Chi-Square	100	93.75
CFS	94.11	100

Fig 4.17 Accuracy of Ensemble before and after GA on Balanced Dataset

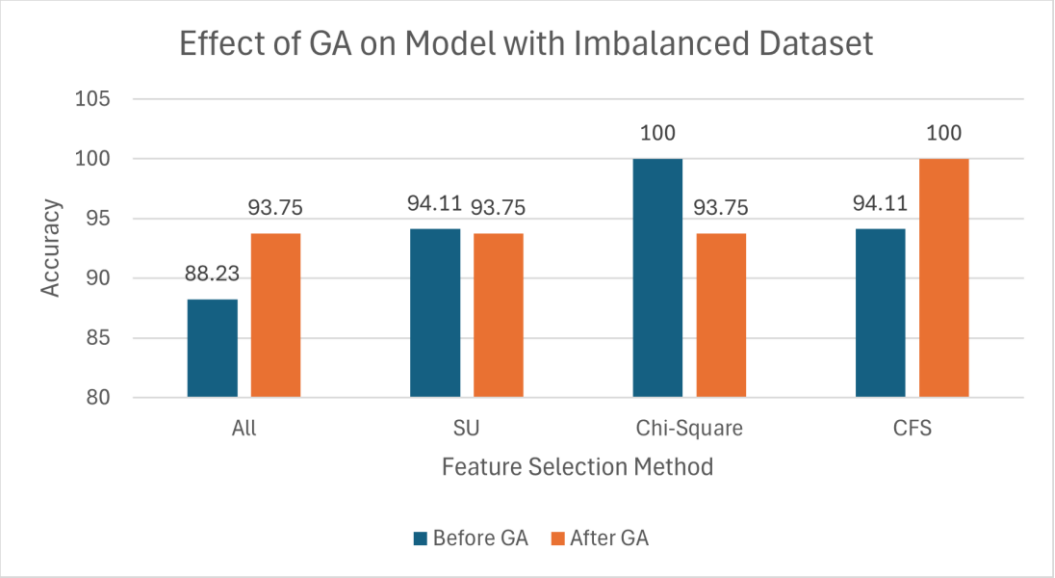

Fig 4.17 Evaluation of Ensemble Model on Balanced Dataset– Effect of GA

Features	All Features		SU		Chi		CFS	
	Imbalanced	Balanced	Imbalanced	Balanced	Imbalanced	Balanced	Imbalanced	Balanced
Before GA	76.92	88.23	76.92	94.11	76.92	100	84.61	94.11
After GA	84.62	93.75	84.62	93.75	76.92	93.75	92.3	100

Table 4.18 Effect of GA on Classification Accuracy of SVM-KNN-RF Ensemble

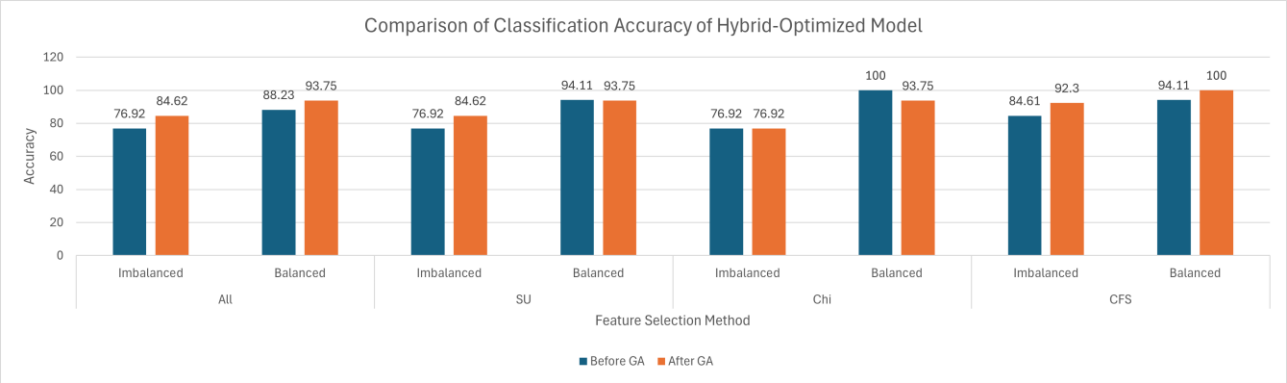

Fig 4.18 Effect of GA on Classification Accuracy of SVM-KNN-RF Ensemble

CONCLUSION AND FUTURE PLANS

It is evident from the results that Traditional ML Models trained on Balanced Dataset exhibit better classification accuracy. It is found that feature selection done using Chi Square and CFS produces better accuracy. On Imbalanced Dataset, SVM with Chi Square Feature Selection produces 92.3% accuracy while it produces 94.11% accuracy on Balanced Dataset.

On the other hand, handling class imbalance does produce better results. With Ensemble models, it is very clear that Model trained with CFS based Feature Selection method is better than other Feature Selection Methods. We can identify that both SVM-RF and SVM-KNN-RF models exhibit equally better classification. But for further optimization, we consider SVM-KNN-RF model due to its better performance even in Imbalanced Dataset.

After optimizing weights of the Soft Voting SVM-KNN-RF Ensemble model using Genetic Algorithm, it is found that the model performs better with CFS Feature Selection. But it is also found that the model is very much susceptible to overfitting and thus careful re-optimization is required and should be done.

REFERENCES

[1] Tambe S.N., Potharaju S., Amiripalli S.S., Tirandasu R.K., Algat Y. *Exploring the Effects of High Dimensionality and Imbalanced Data on Predictive Models for Colon Cancer Detection Using Tree, Rule, and Lazy Learning Techniques (2024).* Journal of Artificial Intelligence and Technology, pp. 342 – 350.

[2] Ben Abdennour G., Gasmi K., Ejbali R. *An Optimal Model for Medical Text Classification Based on Adaptive Genetic Algorithm (2024)*. Data Science and Engineering, pp. 378 – 392.

[3] Hameed S.S., Hassan R., Hassan W.H., Muhammadsharif F.F., Latiff L.A. *The microarray dataset of colon cancer in csv format (2021)*. PLOS ONE.

[4] F. Ketata, Z. A. Masry, N. Zerhouni, S. Yacoub, *Explainable Machine Learning Approach with Augmentation for Mortality Prediction (2023)*. IEEE International Conference on Advanced Systems and Emergent Technologies, pp. 01-06.

[5] F. Rustam, N. Saher, A. Mehmood, E. Lee, S. Washington, I. Ashraf. *Detecting ham and spam emails using feature union and supervised machine learning models. Multimedia Tools and Applications (2023)*. 82. 1-17.

[6] Kambar, M.E.Z.N., Nahed, P., Cacho, J.R.F., Lee, G., Cummings, J., Taghva, K. *Clinical Text Classification of Alzheimer's Drugs' Mechanism of Action (2022*). Proceedings of Sixth International Congress on Information and Communication Technology. Lecture Notes in Networks and Systems, vol 235. Springer, Singapore.

[7] Koppad S, Basava A, Nash K, Gkoutos GV, Acharjee A. *Machine Learning-Based Identification of Colon Cancer Candidate Diagnostics Genes (2022)*. Biology (Basel).;11(3):365.

[8] Chicco, D., Jurman, G. *Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone (2020)*. BMC Med Inform Decis Mak 20, 16.

[9] Gupta P, Chiang SF, Sahoo PK, Mohapatra SK, You JF, Onthoni DD, Hung HY, Chiang JM, Huang Y, Tsai WS. *Prediction of Colon Cancer Stages and Survival Period with Machine Learning Approach (2019)*. Cancers (Basel).;11(12):2007